

Propeller 2 • Application Note P2AN002

CORDIC for Real Work

Rotations, distances, headings, and transcendentals from the P2's hardware math solver

November 2026

Version 1.0.5

What You'll Build

Compute rotations, distances, headings, and trig on the P2 — in a few cycles, with no math library.

A shared foundation — queue an operation, do other work, retrieve the result — then a small catalog of recipes you choose among by what your project needs:

- **Distance & heading** — cartesian-to-polar in one operation (the navigation primitive)
- **Rotate a point** — coordinate rotation about the origin
- **Draw a circle** — polar-to-cartesian, plotted live in a DEBUG window
- **Sine & cosine** — waveform synthesis straight from the angle
- **Fixed-point scale & magnitude** — 64-bit-safe multiply/divide, log and exp
- **Pipeline for throughput** — overlap operations for one result every eight clocks
- **The ceiling** — three-phase motor control as coordinate rotations (reference)

Applies to: P2 (Propeller 2) silicon • Spin2 / PNut, current release • any P2 board, with a DEBUG window and no extra hardware.

Iron Sheep Productions, LLC

P2 Knowledge Base Project

Abstract

The P2 has a hardware math solver — a CORDIC engine, shared by every cog — that turns operations a software library would spend hundreds of instructions on into a single instruction with a fixed, known latency. Rotate a point, convert between cartesian and polar, take a sine, a logarithm, a square root, a full 64-bit multiply or divide: each is one operation you hand to the solver and one you read back. This note shows how to use that solver for real work — distance and heading, rotation, waveform synthesis, fixed-point scaling — and how to keep it busy enough to get a result every eight clocks when you have a lot to compute.

What you'll build: a handful of short, runnable programs — each a complete CORDIC technique — that stream their results to a DEBUG window you can check against exact, known answers.

Prerequisites & Hardware

You should already be comfortable with:

- Reading and writing Spin2, and reading PASM2 (the throughput recipe and the capstone are in assembly).
- Starting code in another cog with COGINIT (the pipelining recipe uses a second cog).
- The P2's 32-bit signed integer model. The CORDIC works in integers and fixed-point; there is no floating-point hardware here.

For the full instruction-level reference — every operand, every flag — this note points to the **PASM2 reference** and the Spin2 documentation rather than reproducing them. It *uses* the solver; it does not re-enumerate it.

Hardware:

- Any P2 board (P2 Edge or P2 Eval).
- **Nothing else.** Every recipe verifies in a DEBUG window against a closed-form answer — the CORDIC computes exact, deterministic math, so you confirm a build by comparing its output to arithmetic you already know, with no bench instruments at all.

The Idea

Most processors compute a sine or a rotation in software: a polynomial, a table, a loop. The P2 instead has the operation built into the hub as a **54-stage pipeline**. You do not call a function — you **issue** an operation, go do other work, and **retrieve** the result a fixed number of clocks later.

That shape — *queue, work, retrieve* — is the one idea under everything in this note. It has three consequences worth holding onto before any detail:

- **The result is not instant, but its timing is exact.** When a cog issues a CORDIC operation, the result is available a fixed **55 clocks** later. You can spend those clocks on anything; the answer will be waiting.

- **Angles are a full 32-bit circle.** The CORDIC measures angle as an unsigned 32-bit number where the whole circle is 2^{32}, so `$0000_0000` is 0°, `$4000_0000` is 90°, `$8000_0000` is 180°, and `$C000_0000` is 270°. There is no degrees-or-radians choice and no range to wrap by hand: angles add and subtract modularly, and overflow *is* wrap-around. This single convention is what makes rotation arithmetic so clean on the P2.
- **Two ways in.** From Spin2 you call built-in methods (`ROTXY`, `XYPOL`, `QSIN`, ...) and the interpreter manages the queue-and-retrieve for you. From PASM2 you issue the instruction (QROTATE, QVECTOR, ...) and read the result yourself with GETQX / GETQY — which is what lets you overlap operations for throughput. Use Spin2 for most work; reach for PASM2 when you are processing a lot of data and want the pipeline full.

How It Works (in brief)

The solver offers eight operations. The full encoding of each lives in the PASM2 reference; this is the working summary.

Operation	PASM2	Does
Multiply	QMUL	32 × 32 → 64-bit unsigned product
Divide	QDIV	64 ÷ 32 → quotient + remainder
Fractional divide	QFRAC	64 ÷ 32 → fractional quotient
Square root	QSQRT	√ of a 64-bit value → 32-bit root
Rotate	QROTATE	rotate (X, Y) by an angle
Cartesian → polar	QVECTOR	(X, Y) → (length, angle)
Integer → $\log_2$	QLOG	32-bit value → 5.27 fixed-point log
$\log_2$ → integer	QEXP	5.27 fixed-point → 32-bit value (antilog)

In PASM2 the pattern is always the same — issue, then read:

```
QMUL    base, scale     ' queue a 32x32 multiply
' ... up to 55 clocks of other work ...
GETQX   prod_lo         ' low  32 bits of the product
GETQY   prod_hi         ' high 32 bits
```

GETQX and GETQY wait for the result if it has not arrived yet, so a single operation needs no timing care — the cog simply stalls until the answer is ready. If you read when nothing was ever queued, the read returns in two clocks and sets the QMT (CORDIC-empty) event, which is how you catch a missing-issue bug.

From Spin2 the same operations are methods that return their results directly, with the queue-and-retrieve hidden:

```
x, y         := ROTXY(x, y, angle)    ' rotate a point
length, angle := XYPOL(x, y)          ' cartesian -> polar
s            := QSIN(length, angle, 0) ' length * sin(angle)
```

Two facts set the throughput ceiling, and Recipe 6 builds on them: a cog can issue one CORDIC operation **every eight clocks** (its hub-access window), and the pipeline is 54 stages deep. Because the pipeline is far longer than the issue interval, a cog can keep **several** operations in flight at once — roughly seven (the 54-stage depth divided by the 8-clock window) — issuing a new one each window while reading the matching earlier result. That is how you get one finished result every eight clocks instead of one every 55.

Choosing a Technique

The recipes below are independent — each is a complete program for one job. Pick by what you need:

If you need...	Use	The operation
Distance and direction between two points	**Recipe 1**	QVECTOR / XYPOL
To rotate a point about the origin	**Recipe 2**	QROTATE / ROTXY
To place points around a circle / arc	**Recipe 3**	POLXY
A sine or cosine wave	**Recipe 4**	QSIN / QCOS
64-bit-safe scaling, magnitude, log/exp	**Recipe 5**	MULDIV64, QVECTOR, QLOG/QEXP
To transform a lot of data fast	**Recipe 6**	any op, pipelined
A real-world reference design	**The Ceiling**	three-phase motor control

Each recipe ends with a ♦ **Verify** that compares the DEBUG output to a number you can work out by hand.

Recipe 1 — Distance & Heading

Use it when you have two points and need *how far apart* and *in what direction* — the navigation primitive behind "steer toward the target," range-and-bearing, and vector magnitude.

A single QVECTOR operation (the Spin2 XYPOL method) converts a cartesian vector into polar form: it returns the vector's **length** and its **angle** at once. Feed it the difference between two points and you have distance and heading from one call.

```
CON
  _clkfreq = 200_000_000

PUB main() | dist, heading, dx, dy
```

continues on next page →

↪ continued from previous page

```
  dx := 3
  dy := 4
  dist, heading := XYPOL(dx, dy)        ' (x,y) -> (length, angle)
  DEBUG("distance=", sdec(dist), "  heading=", uhex_long(heading))


                                                    cordic-distance-heading.spin2
```

How this works. `XYPOL` takes the vector `(dx, dy)` and returns its length and its angle. The length is an ordinary integer in the same units as the inputs; the angle is a 32-bit binary angle (the convention from The Idea). For a real navigation problem `dx`/`dy` are `target - current` on each axis, and the heading drops straight out — no `atan2` library, no quadrant bookkeeping, because the binary angle already spans the full circle.

♦ **Verify:** With `(3, 4)` the DEBUG window shows **distance = 5** — the 3-4-5 triangle — and **heading ≈ $25C8_xxxx**, which is about 53°, exactly atan2(4, 3). Change the inputs to `(0, 100)` and the heading becomes `$4000_0000` (90°, straight up); `(-100, 0)` gives `$8000_0000` (180°).

Recipe 2 — Rotate a Point

Use it when you are turning coordinates: spinning a sprite, re-framing a sensor reading, stepping around an orbit.

QROTATE (the Spin2 `ROTXY` method) rotates a point `(x, y)` about the origin by a binary angle and returns the rotated point.

```
CON
  _clkfreq = 200_000_000

  QTR = $4000_0000              ' 90 degrees as a 32-bit binary angle

PUB main() | x, y
  ' Rotate the point (100, 0) by 90 degrees about the origin.
  x, y := ROTXY(100, 0, QTR)
  DEBUG("x=", sdec(x), "  y=", sdec(y))


                                                    cordic-rotate-point.spin2
```

How this works. `ROTXY` computes the rotation `(x*cos(T) - y*sin(T), x*sin(T) + y*cos(T))` in hardware. Because the angle is a binary angle, a turn is just an add: to rotate a little further each frame, add a constant to the angle and let it wrap. To rotate the *other* way, subtract — or add the two's-complement angle, which is the same thing.

★ **Tip:** `ROTXY` rotates the whole vector, so it is also your fastest way to compute a sine *and* cosine of the same angle scaled by a radius: rotating `(radius, 0)` by the angle gives `(radius*cos(T), radius*sin(T))` in one call.

♦ **Verify:** Rotating `(100, 0)` by 90° gives **`(0, 100)`** — the point swings from the +X axis up to the +Y axis. Rotate by `$8000_0000` (180°) instead and it lands at `(-100, 0)`.

Recipe 3 — Draw a Circle

Use it when you want points laid out around a circle or an arc — a dial, a radar sweep, a round gauge, evenly-spaced positions.

`POLXY` is the inverse of Recipe 1: it takes a **length and an angle** and returns the **cartesian point**. Step the angle in equal increments and you walk the circle. Here the points stream to a DEBUG `PLOT` window so you can watch the circle draw itself.

```
CON
  _clkfreq = 200_000_000

  R       = 100                 ' circle radius
  STEPS   = 64                  ' points around the circle (even)
  ASTEP   = $8000_0000 / (STEPS / 2)   ' one full 2^32 circle / STEPS

PUB main() | i, ang, x, y
  DEBUG(`PLOT myplot size 240 240 origin 120 120 dotsize 4)
  repeat
    repeat i from 0 to STEPS-1
      ang := i * ASTEP                  ' fraction of a full 2^32 circle
      x, y := POLXY(R, ang)             ' (length,angle) -> (x,y)
      DEBUG(`myplot `(x, y))
    waitms(500)
```

cordic-draw-circle.spin2

How this works. A full circle is 2^{32} binary-angle units, so `STEPS` equal steps are `2^32 / STEPS` apart. That quotient fits in 32 bits, but writing the divide directly as `$1_0000_0000 / STEPS` won't compile — the `2^32` *dividend* is a 33-bit literal that overflows a 32-bit constant. Halving both sides avoids the oversized literal: `$8000_0000 / (STEPS / 2)` — half the circle divided by half the steps — yields the same quotient. Each `POLXY(R, ang)` places one point at radius `R`; stepping `ang` around the full range walks the whole circle.

♦ **Verify:** The `PLOT` window draws a clean circle of 64 dots, centered on the origin marker, radius 100. If you see an *ellipse*, your X and Y plot scales differ; if you see an *arc* instead of a full circle, `ASTEP` and `STEPS` disagree.

Recipe 4 — Sine & Cosine

Use it when you need a waveform or an oscillator: tone generation, LFOs, dimming curves, any "smoothly varying" value.

`QSIN` returns `length * sin(angle)`; `QCOS` returns `length * cos(angle)`. Sweep the angle and you have a sine wave, scaled to whatever amplitude you pass as `length`.

```
CON
  _clkfreq = 200_000_000

  AMPL  = 1000
  ASTEP = $0200_0000            ' angle increment per sample

PUB main() | ang, s
  DEBUG(`SCOPE sine size 400 200 samples 256 rate 1 pos 100 100)
  DEBUG(`sine 'sin' -1100 1100 180 20 15 CYAN)
  ang := 0
  repeat
    s := QSIN(AMPL, ang, 0)     ' AMPL * sin(ang); 0 -> full 2^32 circle
    DEBUG(`sine `(s))
    ang += ASTEP
    waitms(2)
```

cordic-sine-cosine.spin2

How this works. The third argument to `QSIN` is the number of steps in a full circle; passing **0** selects the native convention where the angle is the full 32-bit binary angle. `AMPL` scales the output, so the wave swings between `+AMPL` and `-AMPL`. Each loop advances the angle by `ASTEP` and emits one sample to the DEBUG `SCOPE`; a larger `ASTEP` is a higher frequency.

★ **Tip:** For a *quadrature* pair (sine and cosine together — the basis of DDS and IQ work), Recipe 2's `ROTXY` trick gives you both from one rotation; or CALL `QCOS` alongside `QSIN` on the same angle.

♦ **Verify:** The `SCOPE` traces a smooth sine wave peaking at **+1000** and **−1000**. At `ang = $4000_0000` (90°), `QSIN` returns **1000** and `QCOS` returns **0**; at `ang = 0`, `QSIN` is **0** and `QCOS` is **1000**.

Recipe 5 — Fixed-Point Scale & Magnitude

Use it when you are doing arithmetic that overflows 32 bits or needs a fraction — scaling a value by a ratio, taking a magnitude, working in decibels — and you have no floating-point unit.

The CORDIC *is* the P2's wide-integer engine. Three idioms cover most fixed-point work: `MULDIV64` for `(a * b) / c` without overflow, QVECTOR for a magnitude that would overflow if you squared it yourself, and QLOG / QEXP for logarithms.

```
CON
  _clkfreq = 200_000_000

PUB main() | a, b, c, scaled, mag, ang, lg, back
  a := 123_456
  b := 789_012
  c := 1000
  scaled := muldiv64(a, b, c)        ' (a*b)/c, 64-bit intermediate
  mag, ang := XYPOL(a, b)            ' sqrt(a*a + b*b), 64-bit-safe
  lg  := qlog_of(a)                  ' base-2 log, 5.27 fixed-point
  back := qexp_of(lg)                ' antilog round-trips back to ~a
```

continues on next page →

↪ continued from previous page

```
    DEBUG("scaled=", udec(scaled), "  mag=", udec(mag))
    DEBUG("log2=", uhex_long(lg), "  back=", udec(back))

  PRI qlog_of(v) : r
    ORG
      QLOG v
      GETQX r
    end

  PRI qexp_of(v) : r
    ORG
      QEXP v
      GETQX r
    end
```

cordic-fixed-point.spin2

How this works. `muldiv64` multiplies `a` by `b` into a 64-bit intermediate (a QMUL) and divides by `c` (a QDIV fed the 64-bit product), so `123_456 * 789_012` never overflows on the way to the answer. `XYPOL` returns a vector's length, which is `sqrt(a*a + b*b)` computed with a 64-bit intermediate — the safe way to take a magnitude, since `a*a` alone would overflow. QLOG returns a base-2 logarithm in **5.27 fixed-point**: the top five bits are the whole part (the position of the value's most-significant set bit) and the low 27 bits are the fractional mantissa. QEXP is its inverse. The two short inline-PASM helpers expose QLOG/QEXP because Spin2 has no one-line method for them.

■ **Hardware — everything is unsigned.** QMUL, QDIV, QFRAC, and QSQRT treat their operands as *unsigned*. When your data is signed, take the magnitude first, remember the sign, and reapply it to the result. (Recipe 1 and Recipe 2 are already signed — QVECTOR and QROTATE handle signed coordinates.)

♦ **Verify:** The output reads **scaled = 97,408,265** (which is `123_456 * 789_012 / 1000`), **mag = 798,612** (the length of the vector `(123_456, 789_012)`), **log2** whole part **16** (because 123,456 sits between 2^{16} and 2^{17}), and **back ≈ 123,456** — the log/antilog round trip.

Recipe 6 — Pipelining for Throughput

Use it when you have *many* values to transform and one-at-a-time is too slow — filtering a buffer, rotating a point cloud, scaling an array.

A single CORDIC result takes 55 clocks. But because the pipeline is 54 stages deep and a cog can issue every 8 clocks, you do not have to wait 55 clocks *per value*. Issue a batch of operations first (**fill**), then settle into reading one result and issuing one new operation each pass (**steady state**), and finally read out the stragglers (**drain**). Done right, you retire one finished result roughly every eight clocks.

This cog squares an array of values that way — a stand-in for any per-element CORDIC transform:

```
CON
  _clkfreq = 200_000_000
  N    = 8                      ' elements to transform
  FILL = 6                      ' in flight (~6-7: 54-stage/8)

VAR
  LONG inbuf[N]                 ' source values
  LONG outbuf[N]                ' results (follow inbuf in hub)
  LONG done                     ' handshake flag

PUB main() | i
  repeat i from 0 to N-1
    inbuf[i] := 10 + i          ' 10,11,...,17
  done := 0
  COGINIT(NEWCOG, @square_engine, @inbuf)
  repeat until done
  repeat i from 0 to N-1
    DEBUG(udec(inbuf[i]), " squared = ", udec(outbuf[i]))


DAT             ORG

' THE RULE: no hub access inside either CORDIC loop. Block-read the
' inputs, keep fill and drain register-only, block-write the results.
' This shape assumes N >= FILL.
square_engine
                MOV     donep, ptra
                ADD     donep, ##N*8        ' @done

                SETQ    #N-1                ' one burst: hub -> cog
                RDLONG  vals, ptra          ' @inbuf -> vals[0..N-1]

                MOV     ii, #0              ' issue index (cog)
                MOV     jj, #0              ' retire index (cog)

' Fill phase: queue FILL operations. Register-only.
                REP     @.fill_end, #FILL
                ALTS    ii, #vals
                MOV     t, 0-0
```

continues on next page →

↪ continued from previous page

```
                QMUL    t, t                ' queue: t*t
                ADD     ii, #1
.fill_end

' Steady state: retire one result, queue one more. Register-only.
                REP     @.steady_end, #N-FILL
                ALTD    jj, #vals
                GETQX   0-0                 ' result lands in cog RAM
                ADD     jj, #1
                ALTS    ii, #vals
                MOV     t, 0-0
                QMUL    t, t
                ADD     ii, #1
.steady_end

' Drain phase: retire the last FILL results. Register-only.
                REP     @.drain_end, #FILL
                ALTD    jj, #vals
                GETQX   0-0
                ADD     jj, #1
.drain_end

                MOV     outp, ptra
                ADD     outp, ##N*4         ' @outbuf
                SETQ    #N-1                ' one burst: cog -> hub
                WRLONG  vals, outp

                WRLONG  #1, donep
                COGID   t
                COGSTOP t

outp            RES     1
donep           RES     1
ii              RES     1
jj              RES     1
t               RES     1
vals            RES     N
```

cordic-pipeline-throughput.spin2

How this works. Two indices track the pipeline: `ii` counts operations *issued*, `jj` counts results *retired*. The fill phase issues `FILL` multiplies before reading anything, so `FILL` operations are in flight at once. Steady state then reads one result (GETQX, which returns results in issue order) and issues one more, holding the in-flight count steady; the drain retires what is left. `FILL = 6` stays within the roughly seven operations the pipeline can hold.

Notice where the hub traffic is. The inputs arrive in **one** block read before the fill starts, and the results leave in **one** block write after the drain finishes. ALTS / ALTD index the cog-RAM buffer in between, so neither CORDIC loop touches hub memory. Results overwrite their own inputs in that buffer, which is safe because `jj` trails `ii` by `FILL` the whole way.

▲ **Pitfall — keep hub access out of both CORDIC loops.** This is not a micro-optimization; it is the difference between correct results and silently wrong ones. Measured on P2 silicon at 200 MHz: a RDLONG inside the fill loop starts losing results at a fill depth of **2**, and a register-only fill with a WRLONG in the drain starts losing them at **3**. Register-only in both loops, with the hub traffic batched outside, stayed correct through a depth of **7**.

What you get when you break it is a full array of plausible-looking numbers with some fraction stale — no fault, no flag, nothing to notice. Note what that measurement does and does not establish: it locates *where* results are lost, not *why*. Take it as the tested shape — keep hub access out of both loops — rather than as a general law about hub access anywhere near the CORDIC. Deep pipelining itself is not the problem; the depths above stayed correct once both loops were register-only.

▲ **Pitfall — protect a pipelined sequence from interrupts.** The 55-clock gap between issuing and retrieving is a window an interrupt can stretch, and a late read means a result is overwritten by the next one. In PASM2 with interrupts enabled, wrap the juggle in a REP block so the cog cannot be interrupted mid-sequence.

♦ **Verify:** The DEBUG window lists each input and its square — **10 → 100, 11 → 121, … 17 → 289**. The point is not the squares (a plain multiply would do that); it is that all eight flow through the solver overlapped, finishing in roughly the time of one operation plus eight windows rather than eight times 55 clocks.

The Ceiling — Three-Phase Motor Control

The recipes above are foundations. To see them composed into a real product, look at **field-oriented motor control**, the technique behind quiet, efficient brushless and AC motor drives. Its mathematical core is the **Park transform**, which converts three phase currents into a rotating two-axis frame — and that transform is nothing but QROTATE operations.

The forward transform rotates three vectors (one per phase, 120° apart) and sums them. Written for the solver, it is three QROTATE issues with only a cheap angle adjustment between them — the same 120° offset expressed as the binary-angle constant `$5555_5555` — and then the results are read back and combined. It is Recipe 6's pipelining and Recipe 2's rotation in one real loop: the three rotations overlap in the pipeline instead of paying the 55-clock latency three times, and the 120° phase spacing is a modular add because the angle is a full-circle binary value.

This note does not rebuild that design — it is the ceiling, shown to make the point that the recipes here are the actual building blocks of production code, not toys. A complete, working Park/inverse-Park implementation by the P2 community (ManAtWork, **OBEX #2811**) is in the OBEX, listed in the Resources; reading it after this note, you will recognize every move.

Going Further

Feeding the streamer. A common next step is to compute values with the CORDIC and *stream* them out a pin — direct digital synthesis, where a QROTATE or `QSIN` produces each sample and

the streamer paces it to a DAC. The streamer is its own subsystem with its own timing; that composition belongs to the **Streamer Programming Guide**, which owns the streamer mechanics this note would otherwise have to duplicate. Use this note for the math, that guide for the streaming.

The FFT butterfly is one rotation. The core step of a Fast Fourier Transform — the *butterfly*, a complex multiply by a twiddle factor — is exactly a QROTATE: rotate the complex point `(re, im)` by the twiddle angle and the adds and subtracts fall out around it. That turns the solver into an FFT engine, one butterfly per issue; pipelined (Recipe 6), a pass's butterflies overlap. Chip Gracey's compact implementation (*"1024-point FFT in 79 longs,"* Parallax forums, listed in Resources) transforms 1024 samples in about 3 ms at 250 MHz, and a hand-tuned variant in roughly 700 µs. For a production transform — real-valued input, windowing to tame spectral leakage, and the digit-reversal bookkeeping a full FFT needs — a complete open library (James Smith's FFT/IFFT, **OBEX #5361**) is also listed in Resources. If you need spectra rather than single rotations, that is the path.

When *not* to use the CORDIC. The solver is fast, but it is shared and has a 55-clock latency. For a single occasional angle where you need an answer *this instant* and to a coarse tolerance, a small polynomial approximation can win — one P2 compass driver (**OBEX #5278**) deliberately computes `atan2` with a hand-tuned polynomial because, for its one-shot use, that beat queuing a CORDIC operation. The lesson is the general one: the CORDIC shines when you have a *stream* of operations to overlap, or need its full precision; for a lone, loose, latency-sensitive value, weigh the alternative.

See It Work / Verify

Every recipe in this note shares the same kind of check, and it is stronger than a bench measurement: because the CORDIC computes deterministic math, the **expected answer is one you can derive by hand**, and a correct build must match it.

What success looks like. Each program streams a value — a distance, a coordinate, a waveform, an array — to a DEBUG window, and that value equals the closed-form result: `XYPOL(3, 4)` is 5; `ROTXY(100, 0, 90 deg)` is `(0, 100)`; `QSIN` of 90° at amplitude 1000 is 1000; `muldiv64(123_456, 789_012, 1000)` is 97,408,265.

When it doesn't work:

- **A result off in only its lowest bits.** Expected. The trigonometric operations are iterative and do not carry full 32-bit precision, so a value you predicted as exactly `(0, 100)` may read `(0, 99)` or `(1, 100)`. That is the engine's resolution, not a bug — see Pitfalls.
- **A wildly wrong or garbage result.** Almost always a sign or unsigned mistake (Recipe 5's note) or reading the wrong result register — GETQX is the X / length / quotient, GETQY is the Y / angle / remainder.
- **A read that returns instantly and sets the QMT event.** You read a result without issuing an operation first. Check that every GETQX / GETQY is paired with an earlier issue.

- **Nothing in the DEBUG window.** Build with DEBUG enabled (`pnut-ts -d`); without it the `DEBUG()` directives are ignored.

Pitfalls & Notes

▲ **Pitfall — the operations are unsigned where you might assume signed.** QMUL, QDIV, QFRAC, and QSQRT are unsigned. Mixing in a negative operand gives a huge positive one. For signed work, separate magnitude and sign, compute on the magnitude, and reapply the sign. (QVECTOR and QROTATE *are* signed — they are built for coordinates.)

▲ **Pitfall — read each result before the next overwrites it.** Each cog's results sit in a small pipeline; if you issue faster than you read, a result you never retrieved is lost. When pipelining (Recipe 6), keep the issued-minus-retired count within what the pipeline holds, and never let an interrupt stall a pipelined sequence (use REP).

★ **Tip — the angle is the whole 32-bit range, so there is no wrap to manage.** Add or subtract angles freely; overflow is exactly the wrap-around you want. The common offsets are memorable as fractions of 2^{32}: 90° = `$4000_0000`, 120° = `$5555_5555`, 180° = `$8000_0000`, 240° = `$AAAA_AAAA`.

■ **Hardware — the CORDIC is shared, fairly.** All cogs use the one solver through the hub rotation; each cog gets a turn every eight clocks — the interval equals the part's cog count, and the P2X8C4M64P has eight — and no cog can monopolize the solver or corrupt another cog's results. You never lock it or wait on another cog, and neither the eight-clock turn nor the 55-clock latency shrinks when fewer cogs are running.

♦ **Verify — the integer operations are exact, the trigonometric ones are not.** Multiply, divide, and square root are exact (they are real 64-bit integer operations). Rotate, vector, sine, log, and exp are computed iteratively, with the magnitude scale-factor corrected in hardware, so the low few bits of those are approximate. No Parallax source states a bit count for that residual, so measure it for your own tolerance rather than designing to a figure.

Conclusion

The P2's CORDIC turns the math that usually dominates a small processor — trigonometry, rotation, wide multiply and divide, logarithms — into single operations with one shared, simple shape: queue it, do other work, retrieve it. Once that shape is in hand, the recipes are small variations on it: a vector becomes a distance and a heading, an angle becomes a point or a wave, a pair of numbers becomes a 64-bit-safe product or magnitude. And when there is a lot to compute, the same pipeline that imposes a 55-clock latency hands it back as one result every eight clocks. Reach for the CORDIC whenever your code is about to compute geometry or do arithmetic that overflows 32 bits — it is already there, in every cog, waiting for the next operation.

Resources

- **Example code (download).** Every recipe in this note is a complete, `pnut-ts`-compiled program, published together as the **P2AN002 source ZIP** beside this note's PDF — six files, one per recipe, ready to build and run.
- **P2 Assembly Language Reference Manual** (a companion P2 Knowledge Base publication) — the full encoding of every CORDIC instruction (QMUL, QDIV, QFRAC, QSQRT, QROTATE, QVECTOR, QLOG, QEXP, GETQX, GETQY, POLLQMT) and the REP instruction used to protect pipelined sequences.
- **P2 Streamer Programming Guide** (a companion P2 Knowledge Base publication) — for streaming CORDIC-computed samples to pins/DACs (direct digital synthesis).
- **OBEX #2811 — Park Transformation** (ManAtWork): a complete three-phase field-oriented-control transform built from QROTATE, the worked reference behind The Ceiling. Find it by object number at the Parallax OBEX (`obex.parallax.com`).
- **OBEX #2812 — Binary Floating Point Routines (IEEE-32 subset)** (ersmith): a software floating-point and transcendental library built on the CORDIC, a deep example of fixed-point format discipline.
- **OBEX #5278 — QMC5883L/HMC5883/BMM150 compass drivers**: the contrast in "Going Further" — a hand-tuned polynomial `atan2` chosen over the CORDIC for a one-shot use.
- **"1024-point FFT in 79 longs"** (Chip Gracey, Parallax P2 forums): the origin of the CORDIC-butterfly FFT — a complete 1024-point transform in 79 longs (~3 ms at 250 MHz), the source behind "Going Further." Search the Parallax forums by title.
- **OBEX #5361 — FFT/IFFT** (James Smith): the fuller companion to the FFT pointer in "Going Further" — real-input FFT, windowing, and radix-4 digit reversal, all built on the same CORDIC butterfly. Find it by object number at the Parallax OBEX (`obex.parallax.com`).

These are the authoritative primary sources the note's facts trace to. (The companion P2 Knowledge Base manuals named in Resources derive from these same sources.)

1. *Propeller 2 Documentation v35 — Rev B/C Silicon* (Chip Gracey, Parallax Inc.) — the CORDIC Solver section: the 54-stage pipeline, the 55-clock result latency, the hub-window issue rate, and the eight operations and their encodings.
2. *Propeller 2 Datasheet* (Parallax Inc.) — CORDIC Solver functions and throughput.
3. *Spin2 Language Documentation* (Chip Gracey, Parallax Inc.) — the built-in CORDIC methods (`ROTXY`, `POLXY`, `XYPOL`, `QSIN`, `QCOS`, `QLOG`, `QEXP`, `MULDIV64`).

Revision History

The example library `P2AN002-src.zip` is versioned with the note, so a ZIP downloaded beside this PDF matches the code printed here. This note's version is on its cover, and the top row below.

Version	Date	In brief
1.0.5	2026-11-06	The interrupt pitfall is stated for PASM2 code with interrupts enabled
1.0.4	2026-09-10	The eight-clock turn named to the part, and precision stated without an uncited bit count
1.0.3	2026-08-16	Recipe 6 keeps hub access out of both CORDIC loops — measured on silicon
1.0.2	2026-08-08	Licensed CC BY-SA 4.0
1.0.1	2026-07-11	Circle-layout step-size derivation; OBEX attributions matched to the catalog
1.0.0	2026-07-03	First community-review release

What changed in each version, and why, is the note's **changelog** — published beside this PDF in the P2 Knowledge Base document catalog. That record is the authority; this table exists so a reader holding only the PDF can tell which revision they have.

Copyright & License

Copyright © 2026 Iron Sheep Productions, LLC and Parallax Inc.

This work is licensed under the Creative Commons Attribution–ShareAlike 4.0 International License (CC BY-SA 4.0) — you may share and adapt it, including commercially, with attribution and under the same terms. To view the full license, visit `https://creativecommons.org/licenses/by-sa/4.0/`.

Parallax, Propeller, and Spin are trademarks of Parallax Inc. This license grants permissions under copyright only; it does not grant rights to use these trademarks, and adapted or redistributed copies must not imply endorsement by, or official status with, Iron Sheep Productions, LLC or Parallax Inc.

Acknowledgments

- **Parallax Inc.** — for the Propeller 2 and the reference documentation that forms the foundation of this work.
- **Chip Gracey** — for the design of the P2 CORDIC solver and for maintaining the silicon documentation that defines its behavior.

- **The P2 community** — ManAtWork (Park transformation, OBEX #2811) and ersmith (Binary Floating Point Routines, OBEX #2812), whose published code grounded the worked examples and the capstone.